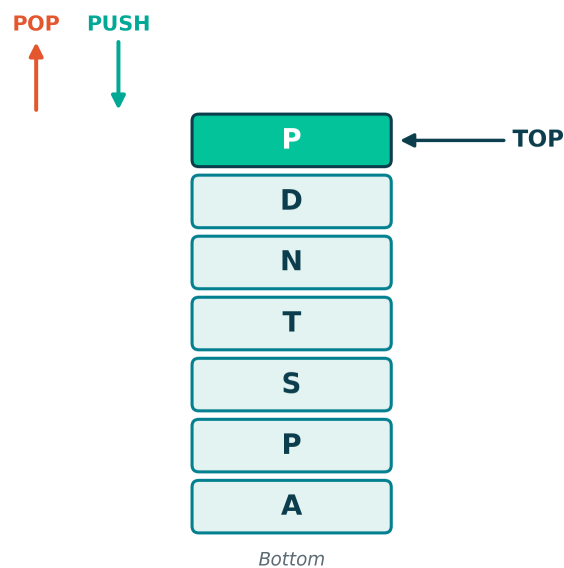


Stack

1. The main characteristic of Stack

A stack is a linear data structure that only ever adds or removes elements at one end, called the top. push places a new element on top of the stack; pop removes and returns the element currently on top. As every insertion and removal happens at the same end, the very last element pushed is always the first one popped — this ordering rule is called LIFO, Last In, First Out.



Your task: The entries in the stack refer to the different layers in networking (TCP/IP). Can you identify what those layers are?

2. The Function Call Stack

Behind every function call, there is a push(.) operation behind the scene. Every return from a function happens through a pop(). The program keeps a function call stack: a stack of “frames”, one per function call currently in progress. When a function calls another function, a new frame is pushed on top; when a function returns, its frame is popped and the program resumes exactly where it left off in the caller.

Your task: Find out what is a “stacktrace”, in connection with program execution.

2.1 Nested Function Calls

```
1  Function f1() {
2      Print(1B)
3      Function f2();
4      Print(1A);
5  }
6  Function f2() {
7      Print(2B);
8      Function f3();
9      Print(2A);
10 }
11 Function f3() {
12     Print(3B);
13     Function f4();
14     Print(3A);
15 }
16 Function f4() {
17     Print(4AB);
18 }
19 f1(); // The first function call being made
```

Your task: What will be the output from the program?

2.2 Reading the Execution Trace

Running f1() prints the following sequence: _____ (Can you find what it prints?). The “B” prints happen on the way in — each function prints its “before” message and then immediately calls the next function, going one level deeper each time. The “A” prints happen on the way back out: once f4() finishes, control returns to f3(), which prints its “after” message and returns to f2(), and so on.

Note the order the functions return in: f4 first, then f3, then f2, then f1 — the exact reverse of the order they were called in. That reversal is the call stack's LIFO behavior in action: each call pushes a new frame, and each return pops the most recently pushed one.

2.3 Visualizing the Call Stack

The figure below shows the call stack configuration when f4 is running. F4 is at the top of the stack, and f1 is the bottom. One stack frame is there for each function that is currently live. Each stack frame mentions the function name, and also the line number from where the execution should continue, after the flow comes back to that function.



Your task: Very deep recursion, or unbounded recursions (missing base case) cause “stack overflow error”. When does this happen, and why is the error named so?

3. Application: Matching Parentheses

A classic use of a stack is checking whether every bracket in an expression is properly opened and closed — and, if so, figuring out which opening bracket each closing bracket belongs to. This comes up constantly in compilers and calculators, wherever nested expressions need to be parsed or validated.

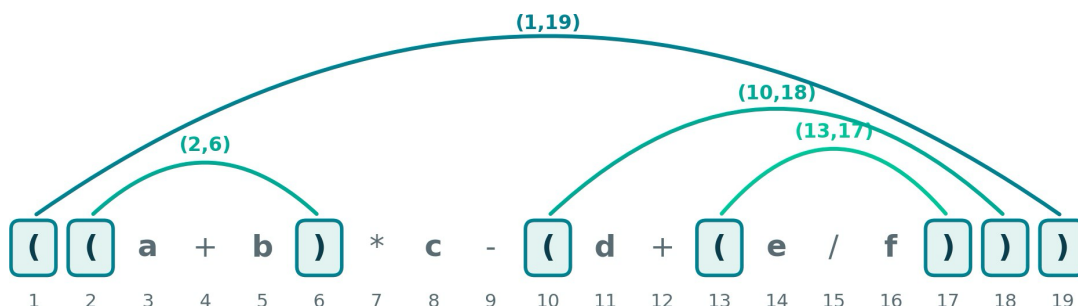
3.1 The Algorithm

1. See an opening bracket “(”. Push its position (index) onto the stack.
2. See a closing bracket “)”. Pop the stack — the popped index is the position of the matching opening bracket.
3. Reach the end of the expression. If the stack is empty, every bracket was matched, and the expression is balanced. If it isn't empty (or a pop is attempted on an empty stack), the expression is unbalanced.

3.2 Worked Example

Suppose we are scanning the expression $((a + b) \times c - (d + (e \div f)))$ from left to right.

The following figure shows the matching brackets.



Matched pairs (open index, close index): (1,19), (2,6), (10,18), (13,17)

The stack empties exactly when the last “)” is processed at position 19 — confirming the expression is fully balanced.

Your task: Find out the stack configuration after consuming each input symbol.

4. Implementing a Stack

The same LIFO behavior can be built two different ways: on top of a fixed-size array, or as a linked list of dynamically allocated nodes. Both implementations support the same four operations — the stack ADT (Abstract Data Type):

`push(x)` : Inserts element `x` on top of the stack

`x = pop()` : Removes the topmost element from the stack and return it

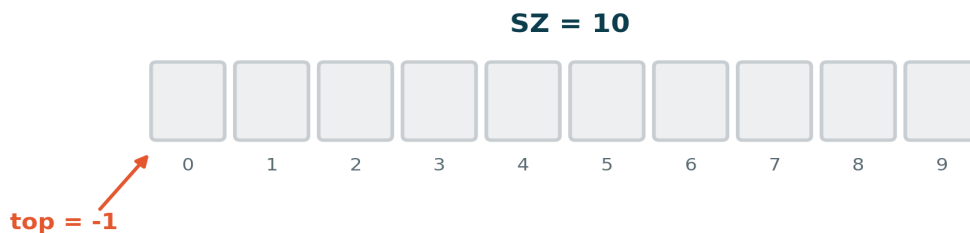
`count()` : Returns the number of elements in the stack

`x = peek()` : Looks at the topmost element without removing it, and returns the element.

`push` and `pop` functionalities are mandatory for stack. The other two functions are optional.

4.1 Array-Based Implementation

The array-based stack uses a fixed-capacity array of size `SZ`, plus an integer `top` that marks the index of the current top element. When the stack is empty, `top = -1`, since there is no valid top element yet:



`push(x)` first checks whether the array is already full — that is, whether `top` has reached the last valid index, `SZ - 1`. If so, it reports a Stack Overflow instead of writing past the end of the array. Otherwise it advances `top` and writes the new value:

```
void push(int x) {
    if (top == SZ - 1) { // Stack Overflow
        return;
    }
    top = top + 1;
    arr[top] = x;
}
```

`pop()` checks the opposite condition — whether `top` is `-1`, meaning the stack is already empty. If so, it reports a Stack Underflow. Otherwise it reads the top value, moves `top` back by one, and returns the value:

```
int pop() {
    int x;
    if (top != -1) {
        x = arr[top];
        top = top - 1;
        return x;
    } else {
        // Stack Underflow
    }
}
```

`count()` and `peek()` are two small helpers built directly from `top`, with no need to touch the rest of the array:

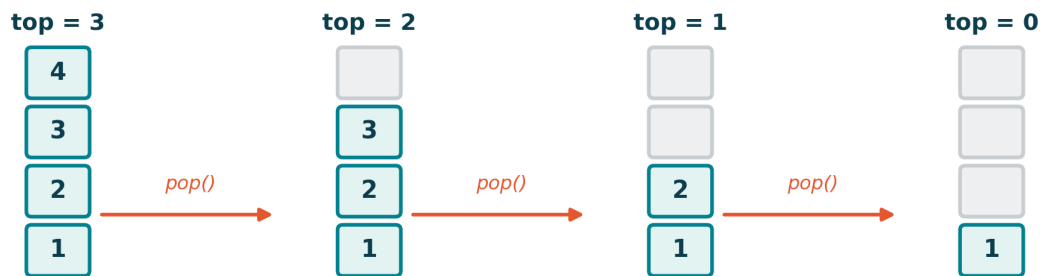
```

int count() {
    return top + 1;
}

int peek() {
    return arr[top];
}

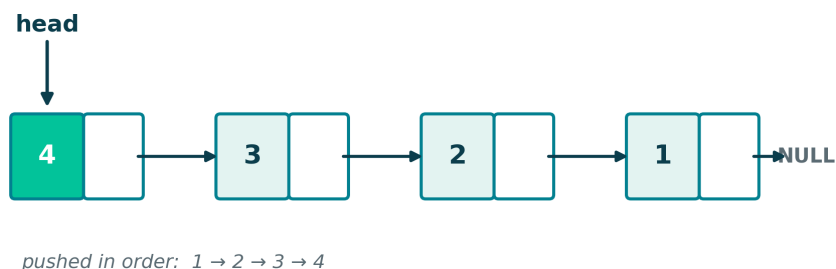
```

Starting from a stack holding 1, 2, 3, 4 (pushed in that order, so `top = 3`), each call to `pop()` removes the top element and moves `top` back by one:



4.2 Linked-List Implementation

A linked-list based stack trades the fixed-size array for a chain of dynamically allocated nodes. Each node stores a value and a pointer to the node below it; a single `head` pointer always references the node currently on top of the stack. `head = NULL` means the stack is empty. Because nodes are allocated on demand, there is no fixed capacity to run out of — the stack simply grows and shrinks node by node:



`push(x)` allocates a new node, sets its value, points it at the current `head`, and then makes it the new `head` — so the newest element is always at the front of the list:

```
void push(int x) {
    Node *n = new Node();
    n->val = x;
    n->next = head;
    head = n;
}
```

`pop()` checks whether `head` is `NULL` (an empty stack) before reading its value and advancing to the next node:

```
int pop() {
    int x;
    if (head != NULL) {
        x = head->val;
        head = head->next;
        return x;
    }
    // Stack Underflow
}
```

5. Summary: Array vs. Linked-List Stack

Array-based: Fixed capacity, decided up front as `SZ` — can overflow if the stack grows past it.

Linked-list based: Dynamic size — grows and shrinks freely, at the cost of one extra pointer stored per element.

`push`, `pop`, `count` and `peek` all run in $O(1)$ constant time, for both array and linked-list-based implementations.